

Обработка и интерпретация сигналов

Конспект лекции 11

Клыков Александр

1 Коды Рида–Соломона

На лекции рассматриваются **коды Рида–Соломона** (РС-коды). Эти коды встречаются во многих практических приложениях и наряду с БЧХ-кодами являются одними из наиболее востребованных в системах хранения и передачи данных.

Когда раньше рассматривались БЧХ-коды, в основном речь шла о двоичных кодах. Однако ничто не запрещает использовать и **недвоичные** коды.

Недвоичный код означает, что в качестве символов берутся не только 0 и 1, а вообще элементы некоторого конечного поля. Это важно и с прикладной точки зрения: когда информация поступает в физический канал, часто используется не двоичная модуляция. В таких системах проверяется не просто присутствие или отсутствие некоторого признака, а выбирается один из нескольких возможных уровней или состояний.

Хотя данные передаются в виде элементов недвоичного алфавита, ошибки внутри одного такого символа могут затрагивать сразу несколько двоичных разрядов. Следовательно, исправление ошибки в одном недвоичном символе означает исправление сразу нескольких двоичных ошибок. В этом смысле коды Рида–Соломона оказываются особенно успешными.

2 Напоминание о БЧХ-кодах

Напомним, как вычисляется порождающая матрица для БЧХ-кода.

Берётся порождающий полином

$$g(x) = \text{lcm}\{M_i(x), i = b, \dots, b + d - 2\},$$

где $M_i(x)$ — минимальные многочлены.

Если

$$g(x) = g_0 + g_1x + g_2x^2 + \dots + g_mx^m,$$

то порождающая матрица строится стандартным образом из коэффициентов $g(x)$ и его последовательных сдвигов.

Соответствующая схема построения приведена на рисунке.

$$G = \begin{bmatrix} g_0 & g_1 & g_2 & \dots & g_m \\ g_0 & g_1 & \dots & g_{m-1} & g_m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ g_0 & g_1 & \dots & g_m \end{bmatrix}$$

n

k

У кода имеется k строк и n столбцов, причём всегда

$$k < n.$$

Избыточность кода равна

$$r = n - k.$$

Чем больше n при фиксированном k , тем больше избыточность. Избыточность хочется уменьшать, потому что чем меньше избыточность, тем эффективнее используется длина кодового слова.

3 Минимальные многочлены и их роль

Чтобы избыточность была меньше, необходимо добиваться того, чтобы степени минимальных многочленов были как можно меньше.

Вспомним, что такое минимальные многочлены. Пусть есть поле

$$GF(p),$$

и полином с коэффициентами из этого поля:

$$a_0 + a_1x + a_2x^2 + \dots, \quad a_i \in GF(p).$$

Далее поле расширяется с помощью некоторого примитивного полинома степени m :

$$p(x), \quad \deg p(x) = m,$$

и получается расширенное поле

$$GF(p^m).$$

В этом расширенном поле существует примитивный элемент α , степени которого порождают все ненулевые элементы поля.

Для каждого элемента расширенного поля можно рассматривать его минимальный многочлен.

Возникает вопрос: каким вообще может быть минимальный многочлен? Многочлен степени меньше 1 быть не может, так как самый простой вариант — это

$$x - \alpha = 0.$$

Но чтобы элемент из расширения был корнем линейного многочлена над исходным полем, он должен принадлежать самому исходному полю. В случае

$$GF(2)$$

это возможно только для элементов 0 и 1.

Именно стремление получить минимальные многочлены как можно меньшей степени и приводит к хорошим кодовым конструкциям.

4 Построение кодов Рида–Соломона

Коды Рида–Соломона строятся над полем

$$GF(q), \quad q = 2^m,$$

по тем же общим правилам, что и БЧХ-коды.

Однако длина у них выбирается равной

$$n = q - 1.$$

Зачем это делается? Позиции кодового слова соответствуют степеням переменной x , а число различных ненулевых элементов поля не должно превышать число элементов поля. В поле $GF(q)$ ненулевых элементов ровно

$$q - 1,$$

поэтому максимальная длина кода:

$$n = q - 1.$$

Для кода Рида–Соломона в качестве порождающего полинома выбирается

$$g(x) = (x - \alpha^b)(x - \alpha^{b+1}) \cdots (x - \alpha^{b+d-2}),$$

где часто берут

$$b = 1,$$

хотя возможны и другие значения.

Недвоичный код, построенный по этим правилам, и называется **кодом Рида–Соломона**.

Если внимательно посмотреть на этот полином, то число множителей равно

$$d - 1.$$

Значит,

$$\deg g(x) = d - 1 = n - k.$$

Следовательно, параметры кода автоматически связаны с заданным минимальным расстоянием.

В смысле минимального расстояния коды Рида–Соломона являются кодами с максимально достижимым расстоянием; во многих случаях они являются оптимальными.

5 Пример построения порождающей матрицы

Рассмотрим расширение

$$GF(2) \rightarrow GF(2^4)$$

с примитивным полиномом

$$p(x) = 1 + x + x^4.$$

Элементы поля задаются таблицей:

степень α	полином	двоичная последовательность
—	0	0000
0	1	0001
1	x	0010
2	x^2	0100
3	x^3	1000
4	$1 + x$	0011
5	$x + x^2$	0110
6	$x^2 + x^3$	1100
7	$1 + x + x^3$	1011
8	$1 + x^2$	0101
9	$x + x^3$	1010
10	$1 + x + x^2$	0111
11	$x + x^2 + x^3$	1110
12	$1 + x + x^2 + x^3$	1111
13	$1 + x^2 + x^3$	1101
14	$1 + x^3$	1001

Тогда

$$n = 15, \quad d = 5.$$

Порождающий полином кода Рида–Соломона:

$$g(x) = (x - \alpha)(x - \alpha^2)(x - \alpha^3)(x - \alpha^4).$$

В поле характеристики 2 знак “минус” совпадает со знаком “плюс”, поэтому можно писать и в виде

$$g(x) = (x + \alpha)(x + \alpha^2)(x + \alpha^3)(x + \alpha^4).$$

После раскрытия скобок:

$$g(x) = x^4 + \alpha_3 x^3 + \alpha_2 x^2 + \alpha_1 x + \alpha_0,$$

где коэффициенты α_i нужно вычислять в поле $GF(2^4)$.

В лекции было отмечено, что вычисление этих коэффициентов — домашнее задание. Для этого кода

$$k = 11, \quad n = 15, \quad d = 5.$$

На прошлых занятиях рассматривался БЧХ-код

$$(15, 7),$$

а теперь получается код

$$(15, 11)$$

на той же длине $n = 15$. Это означает, что число кодовых слов стало больше, а избыточность стала меньше.

6 Декодирование кодов Рида–Соломона

Один из подходов к декодированию уже был рассмотрен раньше: задача сводилась к решению системы уравнений.

Теперь рассматривается алгоритм **Питерсона–Горенштейна–Цирлера**. На самом деле этим алгоритмом можно декодировать и двоичные БЧХ-коды, если просто рассматривать их над подходящим полем $GF(q)$.

Для простоты будем считать, что

$$b = 1,$$

а корни порождающего полинома

$$g(x)$$

равны

$$\alpha, \alpha^2, \dots, \alpha^{d-1}.$$

Пусть кодовое слово:

$$c(x) = c_0 + c_1x + \dots + c_{n-1}x^{n-1}.$$

В канале произошла ошибка:

$$e(x) = e_0 + e_1x + \dots + e_{n-1}x^{n-1}.$$

На выходе из канала получаем

$$v(x) = c(x) + e(x) = v_0 + v_1x + \dots + v_{n-1}x^{n-1}.$$

Так как

$$c(\alpha^j) = 0, \quad j = 1, \dots, d-1,$$

введём синдромы

$$s_j = v(\alpha^j) = c(\alpha^j) + e(\alpha^j) = e(\alpha^j).$$

Предположим, что число ошибок

$$\nu < t, \quad t = \left\lfloor \frac{d-1}{2} \right\rfloor.$$

Пусть ошибки произошли в позициях

$$i_1, i_2, \dots, i_\nu.$$

Тогда

$$s_j = e_{i_1}\alpha^{i_1j} + e_{i_2}\alpha^{i_2j} + \dots + e_{i_\nu}\alpha^{i_\nu j}, \quad j = 1, \dots, d-1.$$

Обозначим

$$Y_h = e_{i_h}, \quad X_h = \alpha^{i_h}.$$

Тогда

$$s_j = Y_1X_1^j + Y_2X_2^j + \dots + Y_\nu X_\nu^j, \quad j = 1, \dots, d-1.$$

Здесь:

- Y_h — значение ошибки;
- X_h — локатор ошибки.

Если $\nu < t$, то такая система имеет единственное решение, но она нелинейна, и число ошибок ν заранее неизвестно.

По сути неизвестными являются величины X_h и Y_h .

7 Полином локатора ошибок

Вместо того чтобы напрямую искать X_h , вводят полином, корнями которого являются величины

$$X_h^{-1}.$$

Определение 1. *Полином*

$$\Lambda(x) = \prod_{j=1}^{\nu} (1 - xX_j)$$

*называется **полиномом локатора ошибок**.*

Если, например,

$$\nu = 3,$$

то есть произошло три ошибки, то система по синдромам имеет вид

$$Y_1X_1 + Y_2X_2 + Y_3X_3 = s_1,$$

$$Y_1X_1^2 + Y_2X_2^2 + Y_3X_3^2 = s_2,$$

$$Y_1X_1^3 + Y_2X_2^3 + Y_3X_3^3 = s_3.$$

Теперь подставим в полином локатора значения

$$X_1^{-1}, X_2^{-1}, X_3^{-1}.$$

Тогда:

$$\Lambda(X_1^{-1}) = 1 + \Lambda_1X_1^{-1} + \Lambda_2X_1^{-2} + \Lambda_3X_1^{-3} = 0,$$

$$\Lambda(X_2^{-1}) = 1 + \Lambda_1X_2^{-1} + \Lambda_2X_2^{-2} + \Lambda_3X_2^{-3} = 0,$$

$$\Lambda(X_3^{-1}) = 1 + \Lambda_1X_3^{-1} + \Lambda_2X_3^{-2} + \Lambda_3X_3^{-3} = 0.$$

Домножим первое уравнение на X_1^3 , второе на X_2^3 , третье на X_3^3 . Получим:

$$X_1^3 + \Lambda_1X_1^2 + \Lambda_2X_1 + \Lambda_3 = 0,$$

$$X_2^3 + \Lambda_1X_2^2 + \Lambda_2X_2 + \Lambda_3 = 0,$$

$$X_3^3 + \Lambda_1X_3^2 + \Lambda_2X_3 + \Lambda_3 = 0.$$

Теперь умножим эти уравнения соответственно на

$$X_1Y_1, \quad X_2Y_2, \quad X_3Y_3$$

и просуммируем.

Получим:

$$X_1^4Y_1 + \Lambda_1X_1^3Y_1 + \Lambda_2X_1^2Y_1 + \Lambda_3X_1Y_1 = 0,$$

$$X_2^4Y_2 + \Lambda_1X_2^3Y_2 + \Lambda_2X_2^2Y_2 + \Lambda_3X_2Y_2 = 0,$$

$$X_3^4Y_3 + \Lambda_1X_3^3Y_3 + \Lambda_2X_3^2Y_3 + \Lambda_3X_3Y_3 = 0.$$

Но по определению синдромов

$$Y_1X_1 + Y_2X_2 + Y_3X_3 = s_1,$$

$$Y_1X_1^2 + Y_2X_2^2 + Y_3X_3^2 = s_2,$$

$$Y_1X_1^3 + Y_2X_2^3 + Y_3X_3^3 = s_3,$$

и аналогично

$$Y_1X_1^4 + Y_2X_2^4 + Y_3X_3^4 = s_4.$$

Следовательно,

$$s_4 + \Lambda_1s_3 + \Lambda_2s_2 + \Lambda_3s_1 = 0.$$

Дальше повторяем ту же операцию, умножая исходные три уравнения на

$$X_1^2Y_1, \quad X_2^2Y_2, \quad X_3^2Y_3,$$

и получаем

$$s_5 + \Lambda_1s_4 + \Lambda_2s_3 + \Lambda_3s_2 = 0.$$

Повторяя ещё раз, получаем

$$s_6 + \Lambda_1s_5 + \Lambda_2s_4 + \Lambda_3s_3 = 0.$$

Итак, система уравнений для коэффициентов полинома локатора ошибок имеет вид

$$\begin{pmatrix} s_1 & s_2 & s_3 \\ s_2 & s_3 & s_4 \\ s_3 & s_4 & s_5 \end{pmatrix} \begin{pmatrix} \Lambda_3 \\ \Lambda_2 \\ \Lambda_1 \end{pmatrix} = \begin{pmatrix} -s_4 \\ -s_5 \\ -s_6 \end{pmatrix}.$$

Все синдромы s_j известны, поэтому эту систему можно решить и найти коэффициенты

$$\Lambda_k.$$

После этого:

- коэффициенты Λ_k дают полином локатора ошибок $\Lambda(x)$;
- находятся корни этого полинома;
- по ним определяются величины X_j ;
- после нахождения X_j решается система на Y_j , то есть на значения ошибок.

Это и есть основная идея алгоритма.

8 Пример: код Рида–Соломона (15, 9)

Рассмотрим код Рида–Соломона

$$RS(15, 9)$$

над полем

$$GF(2^4).$$

Как и раньше, используем расширение

$$GF(2) \rightarrow GF(2^4)$$

с примитивным полиномом

$$p(x) = 1 + x + x^4.$$

Элементы поля задаются таблицей:

степень α	полином	двоичная последовательность
—	0	0000
0	1	0001
1	x	0010
2	x^2	0100
3	x^3	1000
4	$1 + x$	0011
5	$x + x^2$	0110
6	$x^2 + x^3$	1100
7	$1 + x + x^3$	1011
8	$1 + x^2$	0101
9	$x + x^3$	1010
10	$1 + x + x^2$	0111
11	$x + x^2 + x^3$	1110
12	$1 + x + x^2 + x^3$	1111
13	$1 + x^2 + x^3$	1101
14	$1 + x^3$	1001

Код исправляет

3

ошибки, поэтому

$$d = 7.$$

Порождающий полином:

$$g(x) = (x + \alpha)(x + \alpha^2)(x + \alpha^3)(x + \alpha^4)(x + \alpha^5)(x + \alpha^6).$$

Для понимания можно частично выписать коэффициенты. Например, коэффициент при x^5 равен

$$\alpha + \alpha^2 + \alpha^3 + \alpha^4 + \alpha^5 + \alpha^6.$$

Постоянный коэффициент равен произведению всех корней:

$$\alpha \cdot \alpha^2 \cdot \alpha^3 \cdot \alpha^4 \cdot \alpha^5 \cdot \alpha^6 = \alpha^{1+2+3+4+5+6} = \alpha^{21} = \alpha^6,$$

а далее приводится по модулю 15 и по таблице поля.

В лекции было отмечено, что такие вычисления выполняются последовательно для всех коэффициентов.

После построения порождающего полинома выбирается некоторое кодовое слово, в нём вносятся ошибки кратности меньше 3, и на выходе получается новый полином

$$v(x).$$

Далее:

1. вычисляются синдромы

$$s_j = v(\alpha^j), \quad j = 1, \dots, d-1;$$

2. по синдромам строится матрица для коэффициентов Λ_k ;
3. решается система относительно Λ_k ;

4. коэффициенты Λ_k дают полином локатора ошибок $\Lambda(x)$;
5. находятся корни этого полинома;
6. берутся обратные к корням и таким образом находятся локаторы ошибок X_h ;
7. после этого по синдромам решается система уже относительно Y_h , то есть значений ошибок.

Таким образом, алгоритм полностью восстанавливает и позиции ошибок, и их значения.

9 Итог

На лекции было показано:

- как коды Рида–Соломона строятся как недвоичные циклические коды;
- почему для них длина выбирается равной

$$n = q - 1;$$

- как задаётся их порождающий полином;
- как по синдромам строится система для коэффициентов полинома локатора ошибок;
- как работает алгоритм Питерсона–Горенштейна–Цирлера.

Главная идея алгоритма декодирования заключается в том, что вместо прямого поиска локаторов ошибок X_h ищутся коэффициенты полинома

$$\Lambda(x),$$

корнями которого являются величины

$$X_h^{-1}.$$

После этого по найденным локаторам можно определить и сами позиции ошибок, и их значения.